

TD2 : AUTHENTIFICATION API AVEC LARAVEL SANCTUM

Récap :

Q1 : Qu'est-ce qu'un token d'API ?

Réponse : Un token est comme une clé d'hôtel unique donnée à chaque utilisateur.

Q2 : Pourquoi utiliser Sanctum au lieu de sessions ?

Réponse : Les sessions sont pour les navigateurs, les tokens pour APIs/mobiles.

Q3 : Qu'est-ce que le middleware `auth:sanctum` ?

Réponse : C'est le gardien qui vérifie si tu as un token valide.

PARTIE 1 : CRÉER ET PRÉPARER LE PROJET

1 Créer le projet Laravel

```
laravel new auth-api  
cd auth-api
```

Ce que ça fait :

- crée toute l'architecture Laravel
- installe les dépendances de base

2 Configurer la base de données

Fichier : **.env**

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=auth_api  
DB_USERNAME=root  
DB_PASSWORD=
```

Rôle :

- permet à Laravel de se connecter à la base de données

3 Générer la clé de l'application Si ce n'est pas déjà générer

- **php artisan key:generate**

Rôle :

- crée `APP_KEY`
- indispensable pour le chiffrement (mots de passe, tokens)

4 Créer les tables par défaut

php artisan migrate

Résultat :

- table `users` créée

PARTIE 2 : INSTALLER LARAVEL SANCTUM

5 Installer Sanctum

- **composer require laravel/sanctum**

Rôle :

- ajoute l'authentification par token

6 Publier la configuration Sanctum

- **php artisan vendor:publish
--provider="Laravel\Sanctum\SanctumServiceProvider"**

Rôle :

- ajoute la config Sanctum
- ajoute la migration des tokens

7 Lancer les migrations Sanctum

- **php artisan migrate**

Résultat :

- table `personal_access_tokens` créée

PARTIE 3 : CONFIGURER LE MODÈLE USER

Fichier : `app/Models/User.php`

```
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;
}
```

Rôle :

- permet à l'utilisateur de générer des tokens API

PARTIE 4 : CRÉER LE CONTRÔLEUR D'AUTHENTIFICATION

8 Créer le contrôleur

- **php artisan make:controller AuthController**

9 Méthode REGISTER (inscription)

```
public function register(Request $request)
{
    $request->validate([
        'name' => 'required|string',
        'email' => 'required|email|unique:users',
        'password' => 'required|min:6'
    ]);
}
```

```

]);

$user = User::create([
    'name' => $request->name,
    'email' => $request->email,
    'password' => bcrypt($request->password)
]);

return response()->json($user, 201);
}

```

Explication :

- valide les données
- chiffre le mot de passe
- crée l'utilisateur

10 Méthode LOGIN (connexion)

```

public function login(Request $request)
{
    if (!Auth::attempt($request->only('email', 'password'))) {
        return response()->json(['message' => 'Identifiants invalides'], 401);
    }

    $user = User::where('email', $request->email)->first();
    $token = $user->createToken('auth_token')->plainTextToken;

    return response()->json([
        'user' => $user,
        'token' => $token
    ]);
}

```

Explication :

- vérifie email + mot de passe
- génère un token Sanctum
- renvoie le token

Méthode LOGOUT (déconnexion)

```
public function logout(Request $request)
{
    $request->user()->currentAccessToken()->delete();
    return response()->json(['message' => 'Déconnecté']);
}
```

Explication :

- supprime le token actif
- l'utilisateur perd l'accès

PARTIE 5 : ROUTES API

Fichier : **routes/api.php**

```
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

Route::middleware('auth:sanctum')->group(function () {
    Route::get('/profile', function (Request $request) {
        return $request->user();
    });

    Route::post('/logout', [AuthController::class, 'logout']);
});
```

Explication :

- `register` et `login` sont publics
- `profile` et `logout` sont protégés par token

PARTIE 6 : TESTS AVEC POSTMAN

Test 1 : Inscription

- Méthode : **POST**
- URL : `http://127.0.0.1:8000/api/register`
- Body (JSON) :

```
{  
  "name": "Aida",  
  "email": "aida@test.com",  
  "password": "password"  
}
```

✓ Utilisateur créé

Test 2 : Connexion

- POST `/api/login`
- Copier le **token** reçu

Test 3 : Route protégée

- GET `/api/profile`
- Header :

Authorization: Bearer TOKEN

✓ Infos de l'utilisateur affichées

Test 4 : Déconnexion

- POST `/api/logout`
- Header :

Authorization: Bearer TOKEN

✓ Token supprimé

TESTS D'ERREURS

- accès sans token → 401
- mauvais mot de passe → 401
- token supprimé → accès refusé